

Liste diferență

1 Considerente teoretice

În lucrarea de față vom prezenta un nou tip de reprezentare pentru liste. Listele incomplete au fost un pas intermediar spre liste diferență. Dacă în listele incomplete, variabila din coadă era anonimă, în cazul listelor diferență variabila este dată într-un nou parametru.

Un exemplu care ilustrează cum listele diferență ne pot face viața mai ușoară: dacă în listele complete/incomplete pentru a adăuga un element la final trebuie să parcurgem toată lista, în cazul listelor diferență se poate adăuga direct în variabila din coada listei (prin utilizarea acestui nou argument).

1.1 Reprezentare

Listele diferență se reprezintă prin două părți: **începutul** listei și **sfârșitul** listei. De exemplu:

Lista $L=[1,2,3]$ poate fi reprezentată în forma de listă diferență de variabilele:

$S=[1,2,3|X]$ și $E=X$, unde S reprezintă începutul (start), iar E sfârșitul (end)

Denumirea de „diferență” vine de la faptul că lista L poate fi calculată prin diferența dintre S și E .

Lista vidă este reprezentată prin 2 variabile egale ($S=E$). Astfel condiția de oprire se schimbă:

- Pentru liste complete: $\text{pred}(L,...):- L=[], \dots$
- Pentru liste incomplete: $\text{pred}(L,...):- \text{var}(L), !, \dots$
- Pentru liste diferență: $\text{pred}(S, E, ...):- S=E, \text{var}(E), !, \dots$

Exemplu:

$S: [1,2,3,4]$

$E: [3,4]$

$S-E: [1,2]$

$S-E$ (lista diferență) reprezintă lista obținută prin scoaterea părții lui E din S

Nu există avataje când folosim liste diferență ca în exemplul anterior, dar când combinăm conceptele de variabilă liberă și unificare, listele diferență devin unelte utile. De exemplu, lista $[1,2]$ poate fi reprezentată ca lista diferență $[1,2|X]-X$, unde X este o variabilă liberă.

1.2 Predicatul „add”

O listă Prolog este accesată prin primul element și prin coadă. Piedica acestui mod de a vedea lista este faptul că pentru a accesa al n -lea element, trebuie să accesăm toate elementele de dinaintea lui. Dacă, de exemplu, avem nevoie să adăugăm un element la finalul listei, atunci trebuie să trecem prin toate elementele din listă pentru a ajunge la final.

Code

```
add_cl(X, [H|T], [H|R]):- add_cl(X, T, R).  
add_cl(X, [], [X]).
```

Alternativa este să folosim Liste Diferență (reprezentate prin două părți, începutul liste S și finalul liste E):

Code

```
add_dl(X, LS, LE, RS, RE):- RS = LS, LE = [X|RE].  
% variabila LE va conține pe prima poziție elementul adăugat
```

Dacă îl testăm în interpretorul de Prolog ne va da următorul răspuns:

Query

```
?- trace, LS=[1,2,3,4|LE], add_dl(5,LS,LE,RS,RE).  
LE = [5|RE],  
LS = [1,2,3,4,5|RE],  
RS = [1,2,3,4,5|RE]
```

Putem viziona acest proces pas cu pas:

- Inițial avem $\rightarrow LS = [1,2,3,4|LE]$
- Prima operație $RS=LS \rightarrow RS = [1,2,3,4|LE]$
- A doua operație $LE=[X|RE] \rightarrow RS = [1,2,3,4|[X|RE]]$
- Înlocuim X cu valoarea sa numerică $\rightarrow RS = [1,2,3,4|[5|RE]]$
- Simplificăm lista RS $\rightarrow RS = [1,2,3,4,5|RE]$

Pentru a înțelege mai bine modul de funcționare a predicatului, ne putem imagina lista ca fiind reprezentată prin doi "pointeri", unul care arată începutul liste (LS) și al doilea finalul liste (LE), o variabilă fără o valoare atribuită.

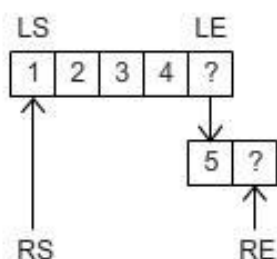


Figura 1. Adăugarea unui element la sfârșitul unei liste diferență

Rezultatul este, de asemenea, reprezentat prin doi "pointeri". Lista rezultat va fi lista de intrare cu un element inserat la final. Începutul liste de intrare și lista rezultată sunt aceeași, deci putem unifica variabila de început a liste de intrare cu variabila de început a liste rezultat ($RS=LS$).

Lista rezultat trebuie să aibă un final, ca și lista de intrare, aceasta va fi într-o variabilă (RE), dar trebuie cumva, să modificăm lista de intrare pentru a adăuga un nou element la final. Deoarece finalul liste de intrare este o variabilă liberă, putem să o unificăm cu lista care începe cu acest nou element și o variabilă nouă, finalul liste rezultat ($LE=[X|RE]$). După finalizarea

execuției predicatului, putem observa ca lista de intrare LS și lista rezultat RS au aceeași valoare, iar finalul listei de intrare nu mai este o variabilă liberă ($LE=[5|RE]$).

1.3 Predicatul „append”

În cazul listelor diferență, predicatul *append* se va scrie pe o singură linie:

Code

```
append_dl(LS1,LE1, LS2,LE2, RS,RE):- RS=LS1, LE1=LS2, RE=LE2.
```

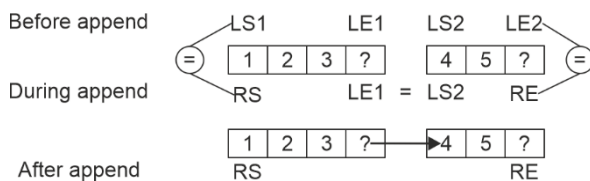


Figura 2. Concatenarea a două liste

Dacă îl testăm în interpretorul de Prolog ne va da următorul răspuns:

Query

```
?- trace, LS1=[1,2,3|LE1], LS2=[4,5|LE2], append_dl(LS1, LE1, LS2, LE2, RS, RE).
LE1 = LS2, LS2 = [4, 5|RE],
LE2 = RE,
LS1 = RS, RS = [1, 2, 3, 4, 5|RE].
```

Putem viziona acest proces pas cu pas:

- Inițial avem $\rightarrow LS1=[1,2,3|LE1], LS2=[4,5|LE2]$
- Prima operație $RS=LS1 \rightarrow RS=[1,2,3|LE1]$
- A doua operație $LE1=LS2 \rightarrow LE1 = [4,5|LE2]$
 - Putem substitui $LE1$ în $RS \rightarrow RS=[1,2,3|[4,5|LE2]]$
- Third operation $RE=LE2$
 - Simplificăm lista $RS \rightarrow RS=[1,2,3|[4,5|RE]]$
- Simplify the RS list using the template $\rightarrow RS = [1,2,3,4,5|RE]$

1.3.1 Sortarea rapidă

Rețineți algoritmul de sortare rapidă (și predicatul): secvența de intrare este împărțită în două – secvența de elemente mai mici sau egale cu pivotul și secvența de elemente mai mari decât pivotul; această procedură este apelată recursiv pe fiecare partiție și secvențele sortate rezultate sunt concatenate cu pivotul pentru a genera secvența sortată:

Code

```
quicksort([H|T], R):-
```

```

partition(H, T, Sm, Lg),
quicksort(Sm, SmS),
quicksort(Lg, LgS),
append(SmS, [H|LgS], R).
quicksort([], []).

```

Ca în cazul predicatului **inorder**, predicatul quicksort/2 va pierde timp de execuție când face append/3 între rezultatele apelurilor recursive. Pentru a evita acest lucru putem folosi liste diferite:

```

Code
quicksort_dl([H|T], S, E):- % s-a adăugat un parametru nou
    partition(H, T, Sm, Lg), % predicatul partition a rămas la fel
    quicksort_dl(Sm, S, [H|L]), %concatenare implicită
    quicksort_dl(Lg, L, E).
quicksort_dl([], L, L). % condiția de oprire s-a modificat

partition(P, [X|T], [X|Sm], Lg):- X<P, !, partition(P, T, Sm, Lg).
partition(P, [X|T], Sm, [X|Lg]):- partition(P, T, Sm, Lg).
partition(_, [], [], []).

```

Predicatul **partition** rămâne neschimbat, scopul acestuia este de a împărți lista în două prin compararea elementelor cu pivotul. Toate elementele din listă trebuie accesate pentru această operație, prin urmare nu putem îmbunătăți performanța acestui predicat.

Predicatul **quicksort** funcționează în aceeași manieră ca înainte: împarte lista în elemente mai mari și mai mici decât pivotul și aplică quicksort recursiv pe fiecare partiție. Diferența între versiunea originală și aceasta este la lista rezultat, reprezentată prin două elemente, începutul și finalul listei, și în consecință modul prin care cele două rezultate ale apelurilor recursive sunt legate cu pivotul (figura de mai jos).

Urmărește execuția la:

```

Query
?- trace, quicksort_dl([4,2,5,1,3], L, []).
?- trace, quicksort_dl([4,2,5,1,3], L, _).

```

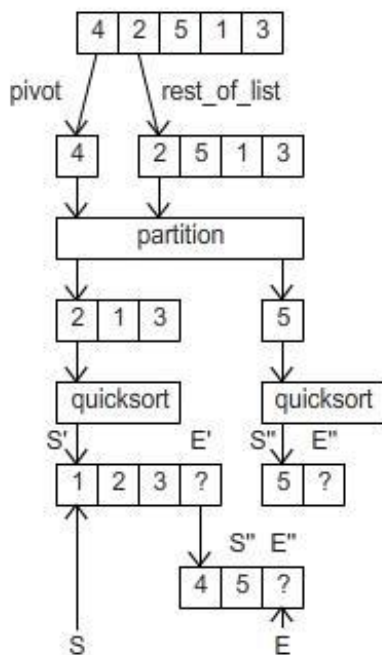


Figura 4. Sortare rapidă folosind liste diferență

1.3.2 Traversare în ordine

Predicatele pentru parcurgerea unui arbore sunt folosite pentru a extrage elementele dintr-un arbore într-o listă într-o ordine specifică. Partea intensiv-computațională a acestor predicate nu este parcurgerea ci combinarea listei rezultat pentru a obține rezultatul final. Cu toate că este ascuns, Prolog va trece prin aceleași elemente ale listei de multiple ori pentru a forma lista rezultat. Putem reduce munca computațională prin înlocuirea listelor simple cu cele diferență.

Predicatul de parcurgere în inordine folosind liste simple pentru a stoca rezultatul:

Code

```

inorder(t(K,L,R),List):-
    inorder(L,ListL),
    inorder(R,ListR),
    append1(ListL,[K|ListR],List).
inorder (nil,[]).
  
```

Prin urmărirea execuției predicatului **inorder/2**, putem observa ușor munca făcută de predicatul append. Este, de asemenea, vizibil că predicatul append accesează aceleași elemente din lista rezultat de mai multe ori în timp ce rezultatele intermediare sunt concatenate la cel final:

Query

```

[... ]
8    3 Exit: inorder(t(5,nil,nil),[5]) ?
12   3 Call: append1([2],[4,5],_1594) ?
13   4 Call: append1([], [4,5],_10465) ?
13   4 Exit: append1([], [4,5], [4,5]) ?
  
```

```

12 3 Exit: append1([2],[4,5],[2,4,5]) ?
[... ]
22 2 Call: append1([2,4,5],[6,7,9],_440) ?
23 3 Call: append1([4,5],[6,7,9],_20633) ?
24 4 Call: append1([5],[6,7,9],_21109) ?
25 5 Call: append1([], [6,7,9],_21585) ?
25 5 Exit: append1([], [6,7,9], [6,7,9]) ?
24 4 Exit: append1([5], [6,7,9], [5,6,7,9]) ?
23 3 Exit: append1([4,5], [6,7,9], [4,5,6,7,9]) ?
22 2 Exit: append1([2,4,5], [6,7,9], [2,4,5,6,7,9]) ?
[... ]

```

Putem îmbunătăți eficiența predicatului **inorder/2** prin înlocuirea vechiului *append* cu operații pe liste diferență. Predicatul **inorder_dl/3** va conține 3 argumente: nodul curent în procesare, începutul listei rezultate și finalul listei rezultate:

Code

```

% când ajungem la finalul arborelui, unificăm începutul și finalul listei
% parțiale de rezultat - lista vidă este reprezentată de 2 variabile egale
inorder_dl(nil,L,L).
inorder_dl(t(K,L,R),LS,LE):-
    % obținem începutul și finalul listelor pentru subarborele stâng și drept
    inorder_dl(L,LSL,LEL),
    inorder_dl(R,LSR,LER),
    % începutul listei rezultat este începutul listei subarborelui stâng
    LS=LSL,
    % cheia K este adăugată între finalul din stânga și începutul din dreapta
    LEL=[K|LSR],
    % finalul listei rezultat este finalul listei subarborelui drept
    LE=LER.

```

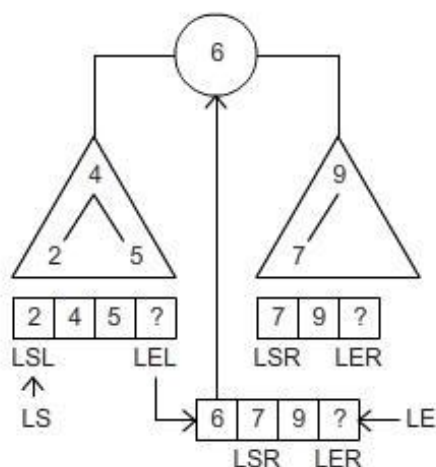


Figura 3. Concatenarea a două liste în traversarea în inordine a unui arbore

Urmărește execuția la:

Query

```
?- trace, tree1(T), inorder_dl(T,L,[]).  
?- trace, tree1(T), inorder_dl(T,L,_).
```

Predicatul poate fi simplificat prin înlocuirea unificărilor explicite cu unificări implicite:

Code

```
inorder_dl(nil,L,L).  
inorder_dl(t(K,L,R),LS,LE):-  
    inorder_dl(L, LS, [K|LT]),  
    inorder_dl(R, LT, LE).
```

2 Exerciții

Înainte de începe exercițiile, adăugați următoarele fapte care definesc arbori (complet și incomplet binar) în script-ul Prolog:

```
Code
% Trees:
incomplete_tree(t(7, t(5, t(3, _, _), t(6, _, _)), t(11, _, _))).
complete_tree(t(7, t(5, t(3, nil, nil), t(6, nil, nil)), t(11, nil, nil))).
```

Scrieți un predicat care:

1. Convertește o listă completă într-o listă diferență și viceversa.

```
Query
?- convertCL2DL([1,2,3,4], LS, LE).
LS = [1, 2, 3, 4|LE]

?- LS=[1,2,3,4|LE], convertDL2CL(LS,LE,R).
R = [1, 2, 3, 4]
```

2. Convertește o listă incompletă într-o listă diferență și viceversa.

```
Query
?- convertIL2DL([1,2,3,4|_], LS, LE).
LS = [1, 2, 3, 4|LE]

?- LS=[1,2,3,4|LE], convertDL2IL(LS,LE,R).
R = [1, 2, 3, 4|_]
```

3. Aplatizează o listă adâncă folosind liste diferență în loc de *append*.

```
Query
?- flat_dl([[1], 2, [3, [4, 5]]], RS, RE).
RS = [1, 2, 3, 4, 5|RE] ;
false
```

4. Traversează un arbore în *pre-ordine* și încă unul pentru *post-ordine* folosind liste diferență în manieră implicită

```
Query
?- complete_tree(T), preorder_dl(T, S, E).
S = [6, 4, 2, 5, 9, 7|E]

?- complete_tree(T), postorder_dl(T, S, E).
```


$S = [2, 5, 4, 7, 9, 6 E]$

5. Colectează toate nodurile care au chei pare, dintr-un arbore binar **complet** folosind liste diferență.

Query

?- complete_tree(T), even_dl(T, S, E).
--

S = [2, 4, 6 E]

6. Colectează toate nodurile care au chei între K1 și K2, dintr-un arbore binar de căutare **incomplet** folosind liste diferență.

Query

?- incomplete_tree(T), between_dl(T, S, E, 3, 7).

S = [4, 5, 6 E]

7. Colectează toate cheile de la o adâncime dată K, dintr-un arbore binar de căutare **incomplet** folosind liste diferență.

Query

? - incomplete_tree(T), collect_depth_k(T, 2, S, E).
--

S = [4, 9 E].
